

## # Quest3 Profile 변경 상세 및 라인별 설명서

기준 브랜치: `Josh\_No\_AI`

구현 커밋: `38d209755a259636b383cd6506a760b3d6205559`

커밋 메시지: `fix(3Tier): map authenticated users to internal profiles`

이 문서는 위 커밋에서 변경한 모든 코드 파일을 현재 줄 번호 기준으로 설명한다. 삭제된 코드는 현재 파일에 줄이 위치”와 “현재 대체 코드 위치”를 함께 기록한다.

### ## 1. 변경 파일 전체 목록

코드 변경 파일은 총 10개다.

1. `module/quest3Tier/func/repository/model/Profile.mjs`
2. `module/quest3Tier/func/service/profile.mjs`
3. `module/quest3Tier/func/handler/handlerWrapper.mjs`
4. `module/quest3Tier/func/handler/handler.mjs`
5. `module/quest3Tier/func/route.mjs`
6. `module/quest3Tier/func/deploy/db/migration/20260724000000-create-profile-table.mjs`
7. `module/quest3Tier/func/deploy/db/migration/20260726000000-link-profile-foreign-keys.mjs`
8. `module/quest3Tier/ui/api/question.ts`
9. `module/quest3Tier/ui/AppRoutes.tsx`
10. `module/quest3Tier/func/vitest/profile.test.mjs`

문서 파일은 다음 2개가 최초 구현 커밋에 추가됐다.

11. `Documentation/Quest3\_Profile\_구현\_설명서.md`
12. `Documentation/Quest3\_Profile\_구현\_설명서.pdf`

### ## 2. 변경의 핵심 결론

변경 전에는 Microsoft JWT의 `oid`를 Quest3 Profile의 `internal\_id`처럼 사용했다.

변경 후에는 다음과 같이 역할을 분리한다.

```
```text
decoded.oid
  → Microsoft 외부 사용자 ID
  → Profile.external_id 검색에만 사용

profile.internal_id
  → Quest3 내부 Profile ID
  → request.userData.profileId로 downstream handler에 전달
```
```

같은 `external\_id`에 여러 Profile이 있으면 `createdAt ASC`, `internal\_id ASC` 기준 첫 번째 file을 사용한다.

### ## 3. Profile 모델 변경

파일: `module/quest3Tier/func/repository/model/Profile.mjs`

현재 관련 위치: 10~19행

### 변경 전

```
```js
external_id: {
```

```
    type: DataTypes.UUID,  
    allowNull: false,  
    unique: true,  
  },  
},
```

### 변경 후: 현재 16~19행

```
```js  
external_id: {  
  type: DataTypes.UUID,  
  allowNull: false,  
},  
},
```

### 정확히 제거한 코드

```
```js  
unique: true,
```

### 이유

Jake의 요구사항은 한 Microsoft `external\_id`에 여러 Quest3 `internal\_id`가 연결될 수 있다는 것. unique constraint가 있으면 두 번째 Profile 생성 시 DB 오류가 발생하므로 제거했다.

### 그대로 유지한 코드: 현재 10~15행

```
```js  
internal_id: {  
  type: DataTypes.UUID,  
  allowNull: false,  
  primaryKey: true,  
  defaultValue: DataTypes.UUIDV4,  
},  
},
```

`internal\_id`는 Quest3가 자동 생성하는 primary key다. Microsoft `oid`를 이 값으로 직접 지정하지 않는다.

## 4. Profile 서비스 변경

파일: `module/quest3Tier/func/service/profile.mjs`

현재 구현 전체 위치: 6~32행

### 4.1 모델 접근 방식

현재 6행:

```
```js  
import Model from "../repository/model/index.mjs";
```

### 이유

production에서 실제 `.mjs` 모델들을 초기화하는 repository model index를 사용한다. DI container의 models loader는 현재 `.js` 파일만 검색하기 때문에 `Profile.mjs`를 제공하지 못한다. 테스트에서는 `index.js`로 repository model index 자체를 교체한다.

### 4.2 입력 검증: 현재 9~14행

```
```js
```

```

async function ensureProfile(externalId) {
  if (!externalId || !isUuid(externalId)) {
    const error = new Error("Authenticated profile ID is required");
    error.status = 401;
    throw error;
  }
}

```

이 부분은 외부 ID가 없거나 UUID가 아니면 DB 접근 전에 401 오류를 던진다.

### 4.3 조회 기준 변경: 현재 16~22행

변경 전:

```

```js
const [profile, created] = await Model.Profile.findOrCreate({
  where: { internal_id: externalId },
  defaults: {
    internal_id: externalId,
    external_id: externalId,
  },
});

```

문제점:

1. JWT `oid`를 `internal\_id`에서 검색했다.
2. 새 Profile 생성 시 internal ID와 external ID를 같은 값으로 강제했다.
3. 여러 Profile 중 어느 것을 선택할지 나타내지 못했다.

변경 후 현재 17~23행:

```

```js
const existingProfile = await Profile.findOne({
  where: { external_id: externalId },
  order: [
    ["createdAt", "ASC"],
    ["internal_id", "ASC"],
  ],
});

```

각 줄의 의미:

- 17행: 한 행을 찾는 `findOne` 실행
- 18행: Microsoft `oid`와 같은 `external\_id` 검색
- 19~22행: 가장 먼저 생성된 Profile을 첫 번째로 정렬
- 생성 시간이 같으면 `internal\_id`가 작은 행을 선택해 결과를 결정적으로 유지

SQL 개념:

```

```sql
SELECT *
FROM profile
WHERE external_id = :externalId
ORDER BY "createdAt" ASC, internal_id ASC
LIMIT 1;

```

### 4.4 기존 Profile 반환: 현재 25~27행

```

```js

```

```
if (existingProfile) {
  return { profile: existingProfile, created: false };
}...
```

기존 Profile이 있으면 새 행을 만들지 않는다.

### 4.5 신규 Profile 생성: 현재 29~30행

```
```js
const profile = await Profile.create({ external_id: externalId });
return { profile, created: true };
```
```

`external\_id`만 전달한다. `internal\_id`는 Profile 모델의 UUIDV4 기본값이 만든다.

변경 전의 다음 충돌 검사도 제거했다.

```
```js
if (profile.external_id !== externalId) {
  // 409 error
}...
```

이전 구조는 internal ID와 external ID가 같다는 전제를 검사했다. 새 구조에서는 두 ID가 원래 서로 다른  
사가 필요하지 않다.

## 5. 인증 wrapper 변경

파일: `module/quest3Tier/func/handler/handlerWrapper.mjs`

현재 관련 위치: 43~59행

### 변경 전

```
```js
const decoded = await provider.decode(token);
const profileId = decoded.oid;
const { profile, created: profileCreated } = await ensureProfile(profileId);
user = { profileId, profile, profileCreated };
```
```

문제점: Microsoft `oid`를 그대로 Quest3 `profileId`로 downstream에 전달했다.

### 변경 후: 현재 52~56행

```
```js
const decoded = await provider.decode(token);
const externalId = decoded.oid;
const { profile, created: profileCreated } = await ensureProfile(externalId);
const profileId = profile.internal_id;
user = { profileId, externalId, profile, profileCreated };
```
```

라인별 의미:

- 52행: JWT 검증 및 claim decode
- 53행: `oid`를 Microsoft 외부 ID로 명시
- 54행: 외부 ID에 연결된 Profile 조회 또는 생성
- 55행: 선택된 Profile의 내부 ID를 실제 `profileId`로 결정
- 56행: downstream에 전달할 인증 사용자 데이터 구성

현재 59행:

```
```js
request.userData = user;
```

이후 handler는 `request.userData.profileId`를 사용하며 이 값은 `profile.internal\_id`다.

## ## 6. Handler 변경

파일: `module/quest3Tier/func/handler/handler.mjs`

### ### 6.1 삭제: EnsureProfile handler

변경 전 파일 8~19행에 있던 다음 함수 전체를 제거했다.

```
```js
async function EnsureProfile(request) {
  const { profile, profileCreated: created } = request.userData;
  return {
    status: created ? 201 : 200,
    return: {
      internalId: profile.internal_id,
      externalId: profile.external_id,
      created,
    },
  };
}
```

파일 마지막 default export의 `EnsureProfile` 항목도 제거했다. 현재 export 목록은 854행부터 시작한다.

제거 이유: 프론트엔드가 `POST /profile`로 초기화하지 않고 모든 인증 요청의 wrapper가 자동 처리하기 때문이다.

### ### 6.2 질문 생성: 현재 57~61행

```
```js
const profileId = request.userData.profileId;
```

질문 생성자의 ID는 request body가 아니라 인증 wrapper가 선택한 내부 ID다.

### ### 6.3 답변 생성: 현재 212~217행

```
```js
const { id: questionId } = request.params;
const profileId = request.userData.profileId;
const { answer = null, option = null, duration } = request.clientParams;
```

질문 ID와 답변 내용은 클라이언트 요청에서 받지만 답변 작성자 ID는 인증 데이터에서 받는다.

### ### 6.4 질문 목록: 현재 321~324행

변경 전:

```
```js
const { profileId } = request.params;
```

변경 후:

```
```js
const profileId = request.userData.profileId;
```

사용자가 URL의 Profile ID를 바꿔도 서버는 그 값을 현재 사용자로 신뢰하지 않는다.

### 6.5 질문 공유: 현재 461~466행

```
```js
const senderId = request.userData.profileId;
const { receiverIds = [] } = request.clientParams;
```

sender는 현재 로그인 Profile이므로 인증 데이터에서 가져오고, receiver는 사용자가 선택하는 공유 대상이다.

### 6.6 공유 질문 목록: 현재 529~532행

변경 전:

```
```js
const { profileId } = request.params;
```

변경 후:

```
```js
const profileId = request.userData.profileId;
```

공유 질문 목록도 URL 값 대신 인증된 내부 ID로 조회한다.

### 6.7 질문 Patch: 현재 597~601행

```
```js
const profileId = request.userData.profileId;
```

질문 변경 기록인 QuestionAction 작성자를 인증된 Profile로 기록한다.

### 6.8 Follow-up command: 현재 823~840행

```
```js
const profileId = request.userData.profileId;
const body = { ...request.clientParams, profileId };
```

body에 `profileId`가 들어와도 spread 이후 인증된 값으로 덮어쓴다. FollowUpCmd sender 및 questionId는 sender에 같은 내부 ID를 전달한다.

### 6.9 Share command: 현재 843~851행

```
```js
const profileId = request.userData.profileId;
const body = { ...request.clientParams, profileId };
```

QuestionShareCmd sender도 인증된 내부 ID다.

## 7. Route 변경

파일: `module/quest3Tier/func/route.mjs`

### 제거한 route

변경 전 13~19행:

```
```js
app.http("EnsureProfile", {
  route: "profile",
  methods: ["POST"],
  authLevel: "anonymous",
  handler: requestHandler(questionHandler.EnsureProfile),
});
```

현재 파일은 import 다음 13행에서 바로 `CreateQuestion` route가 시작한다.

제거된 HTTP API:

```
```text
POST /profile
```
```

현재 각 route는 계속 `requestHandler(...)`로 감싸져 있다. 예를 들어 CreateQuestion은 현재 13~14행, GetQuestion은 15~16행, UpdateQuestion은 17~18행, DeleteQuestion은 19~20행, GetProfile은 21~22행, UpdateProfile은 23~24행, DeleteProfile은 25~26행, CreateQuestion은 27~28행, GetQuestion은 29~30행, UpdateQuestion은 31~32행, DeleteQuestion은 33~34행, GetProfile은 35~36행, UpdateProfile은 37~38행, DeleteProfile은 39~40행이다. 그러므로 별도 초기화 API 없이도 인증 요청 시 wrapper가 필요하다.

## 8. Profile 테이블 migration 변경

파일: `module/quest3Tier/func/deploy/db/migration/20260724000000-create-profile-table.mjs`

### 변경 전 external\_id

```
```js
external_id: {
  allowNull: false,
  type: Sequelize.UUID,
  defaultValue: Sequelize.UUIDV4,
},
```

### 변경 후: 현재 17~20행

```
```js
external_id: {
  allowNull: false,
  type: Sequelize.UUID,
},
```

제거한 줄:

```
```js
defaultValue: Sequelize.UUIDV4,
```

이유: `external\_id`는 DB가 임의 생성할 값이 아니며 검증된 Microsoft JWT `oid`를 반드시 받아야 한다.

현재 11~16행의 `internal\_id` UUIDV4 기본값은 유지한다. 자동 생성 대상은 내부 ID이기 때문이다.

## 9. Profile 외래 키 migration 변경

파일: `module/quest3Tier/func/deploy/db/migration/20260726000000-link-profile-foreign-keys.mjs`

### 현재 8~20행: 연결 대상 11개

```
```text
question.profileId
questionAnswer.profileId
questionShare.senderProfileId
questionShare.receiverProfileId
logQuestion.profileId
questionAction.profileId
followUpCmd.senderProfileId
followUpFilter.senderProfileId
followUpEvent.senderProfileId
questionShareCmd.senderProfileId
questionShareEvent.senderProfileId
```
```

### 현재 29~37행: 기존 데이터 backfill

기존 테이블에서 사용 중인 Profile ID를 먼저 `profile` 테이블에 넣는다. 그래야 현재 39~52행에서 외래 대상이 존재한다.

### 제거한 unique constraint

변경 전 up migration에는 다음 코드가 있었다.

```
```js
await queryInterface.addConstraint("profile", {
  fields: ["external_id"],
  type: "unique",
  name: "profile_external_id_unique",
  transaction,
});
```

down migration에는 이를 제거하는 코드가 있었다.

```
```js
await queryInterface.removeConstraint(
  "profile",
  "profile_external_id_unique",
  { transaction }
);
```

두 부분 모두 삭제했다. 하나의 external ID에 여러 Profile을 허용하기 위해서다.

현재 up migration은 backfill 후 바로 39행의 외래 키 반복문으로 이동한다. 현재 down migration은 52행의 외래 키들만 역순 제거한다.

주의: 수정 전 migration이 이미 배포된 DB에는 unique constraint가 남아 있다. 그런 DB에는 새로운 constraint를 추가하기 위해 제거 migration이 별도로 필요하다.

## 10. 프론트엔드 API 변경

파일: `module/quest3Tier/ui/api/question.ts`

### 제거한 함수

변경 전 10~24행:



```

```ts
export const ensureCurrentProfile = async () => {
  await loadConfig();
  const apiDomain = getConfig("QUEST3TIER_DOMAIN");
  const response = await jwtFetch(`${apiDomain}/profile`, {
    method: "POST",
  });

  if (!response.ok) {
    throw new Error("Failed to initialise the current Q3 profile");
  }

  const data = await response.json();
  return data.return;
};

```

이 함수 전체를 제거했다. 현재 파일은 import 다음 10행부터 `getQuestionsByUser()` 설명이 시작한다.

### 그대로 남아 있는 legacy URL: 현재 11~18행

```

```ts
const profileId = localStorage.getItem("profileId");
const response = await jwtFetch(
  `${apiDomain}/profile/${profileId}/question`,
  { method: "GET" }
);

```

URL parameter는 아직 남아 있지만 backend handler는 이 값을 현재 사용자로 신뢰하지 않고 `request.profileId`를 사용한다. 향후 `/me/question` 같은 URL로 변경할 수 있는 legacy 구조다.

## 11. 프론트엔드 AppRoutes 변경

파일: `module/quest3Tier/ui/AppRoutes.tsx`

### 제거한 import

```

```ts
import { ensureCurrentProfile } from "../api/question";

```

### 제거한 state

```

```ts
const [profileReady, setProfileReady] = useState(false);
const [profileError, setProfileError] = useState<Error | null>(null);

```

### 제거한 effect

`ensureCurrentProfile()`을 호출하고 성공하면 `profileReady`를 true로 만들며 실패하면 `profile`를 저장하던 effect 전체를 제거했다.

### 반환 변경

변경 전:

```

```tsx
return profileReady ? routes : null;

```

변경 후 현재 74행:

```
```tsx
return routes;
```

효과: Profile 초기화 HTTP 요청이 끝날 때까지 빈 화면을 보여주지 않는다. 실제 API 요청이 wrapper를 통해 자동 준비한다.

현재 8행의 ``useEffect``, ``useState`` import는 Profile 초기화용이 아니라 ``RouteModuleElement`` loader 상태 관리에 계속 사용되므로 제거하지 않았다.

## ## 12. 테스트 변경

파일: ``module/quest3Tier/func/vitest/profile.test.mjs``

현재 테스트 위치: 11~104행

### ### 12.1 Mock 변경: 현재 11~20행

변경 전에는 ``findOrCreate`` 하나만 mock했다.

변경 후:

```
```js
const findOne = vi.fn();
const create = vi.fn();
```

실제 테스트에서는 ``vi.hoisted()``로 두 mock을 만든 뒤 현재 16~20행의 ``vi.mock()``으로 ``repository/index.mjs``의 ``Profile``을 교체한다. Profile 서비스가 조회 후 필요할 때 생성하므로 두 함수를 각각 production과 같은 import 경로를 유지하면서 DB 호출만 mock한다.

### ### 12.2 신규 Profile 생성 테스트: 현재 23~41행

서로 다른 UUID를 사용한다.

```
```text
externalId = Microsoft oid
internalId = Quest3가 생성한 Profile ID
```

검증 내용:

- ``external_id``로 조회
- ``createdAt``, ``internal_id`` 정렬
- 기존 행이 없으면 ``external_id``만 넣어 생성
- 반환된 `internal ID`가 별도 값
- ``created``가 true

### ### 12.3 기존 첫 Profile 재사용: 현재 43~56행

``findOne``이 반환한 Profile을 그대로 재사용하고 ``create``가 호출되지 않는지 검증한다.

### ### 12.4 ID 누락: 현재 58~62행

401 오류와 DB 미호출을 검증한다.

### ### 12.5 잘못된 UUID: 현재 64~68행

``not-a-uuid`` 입력에 401 오류가 발생하고 DB를 호출하지 않는지 검증한다.

### 12.6 wrapper internal ID 전달: 현재 70~103행

auth provider는 external ID를 반환하고 Profile 조회 결과는 서로 다른 internal ID를 반환한다.

최종 assertion:

```
```js
return: {
  profileId: internalId,
  profileCreated: false,
}
```
```

이 테스트가 `decoded.oid`가 아니라 `profile.internal\_id`가 downstream에 전달됨을 증명한다.

### 테스트에서 제거한 이전 409 케이스

이전에는 internal ID와 external ID가 같아야 한다는 전제로 두 값이 다르면 409를 검사했다. 새 설계에서 것이 정상이라 이 테스트를 제거했다.

## 13. 실행한 검증

### Profile 단위 테스트

실행 위치:

```
```text
/Users/healer/Documents/GitHub/ZBReactArchitecture/未命名
```
```

명령:

```
```text
pnpm --filter quest3Tier test:profile
```
```

결과:

```
```text
Test Files  1 passed (1)
Tests       5 passed (5)
```
```

이 테스트는 mock 기반 단위 테스트이며 실제 PostgreSQL migration 통합 테스트는 아니다.

### 추가 검사

```
```text
pnpm --dir module/quest3Tier/ui exec tsc --noEmit
node --check <변경된 .mjs 파일들>
git diff --check
```
```

변경 대상 Quest3 UI TypeScript 검사, Node 문법 검사, whitespace 검사가 통과했다.

전체 workspace `typecheck` 명령은 같은 package name의 다른 모듈까지 선택하고 기존 React Router 설정을 찾지 못해 실패했으며, 변경 대상 Quest3 UI는 직접 실행한 TypeScript 검사에서 통과했다.

## 14. 발표 시 코드 순서

다음 순서로 파일을 열면 흐름을 가장 쉽게 설명할 수 있다.

1. `Profile.mjs` 10~19행: 두 ID의 DB 정의
2. `profile.mjs` 9~30행: external ID로 첫 Profile 조회/생성
3. `handlerWrapper.mjs` 52~59행: oid를 external ID로 받고 internal ID로 변환
4. `handler.mjs` 57~60행: 실제 handler가 내부 ID 사용
5. `route.mjs` 13~18행: 모든 route가 wrapper를 통과
6. `AppRoutes.tsx` 62~74행: 프론트 초기화 API 없이 렌더링
7. `profile.test.mjs` 70~103행: 서로 다른 두 ID로 핵심 동작 증명

## ## 15. 30초 발표 문장

“JWT의 oid는 Microsoft 외부 사용자 ID이고 Quest3 internal\_id는 내부 Profile primary key입니다. 같은 Microsoft 사용자에게 여러 Profile이 연결될 수 있으므로 external\_id의 unique 제약을 제거했습니다. questHandler가 oid를 external\_id로 조회하고, 여러 결과 중 생성 순서상 첫 Profile을 선택하며, 없으면 internal UUID를 가진 Profile을 생성합니다. 이후 모든 downstream handler에는 profile.internal\_id를 quest.userData.profileId로 전달합니다. 이 처리가 wrapper에서 자동으로 이루어지므로 프론트와 백엔드의 profile 초기화 API를 제거했습니다.”

## ## 16. 현재 남은 후속 사항

1. 실제 공유 DB에 이전 unique constraint가 적용됐는지 확인해야 한다.
2. 적용됐다면 새 migration으로 `profile\_external\_id\_unique`를 제거해야 한다.
3. legacy `/profile/{profileId}/...` URL은 향후 `/me/...` 형태로 정리할 수 있다.
4. Swagger에 남은 request body `profileId` 설명을 실제 구현에 맞게 정리해야 한다.
5. 같은 사용자의 최초 요청이 완전히 동시에 들어오면 Profile이 두 개 생성될 가능성이 있다. 현재 복수 Profile이 생성인지 제품 정책 확인이 필요하다.